

Institut
Teknologi
Sepuluh Nopember

Graph Theory

Midterm - Task 2: Directed Chinese Postman Problem (DLH Surabaya Dump Truck Simulator)

Work realised by Melvyn, Adam and Hugo

Teacher: Ilham Gurat Adillion

Date: 10/2026

Level: 3rd Semester

Table des matières

Table des matières 2
1. Algorithm Idea 3
2. Implementation 3
3. Computational Complexity 4

1. Algorithm Idea

The goal of this task is to find the most efficient route for a dump truck to traverse every street in a selected operational radius and return to its starting base. Because urban road networks contain one-way streets and divided roads, this cannot be modeled as a standard undirected Chinese Postman Problem (CPP). Instead, it must be formulated as a Directed Chinese Postman Problem (dCPP) on a directed multigraph G = (V, E), where V represents the intersections and E represents the directed road segments.

The algorithm relies on three main theoretical steps:

1. **Strongly Connected Component (SCC) Pruning**: A closed tour that traverses every edge and returns to the start exists if and only if all edges belong to the same strongly connected component. Any dead-end or one-way street pointing outside the operational radius without a return path will trap the truck. Therefore, the first step is to extract the largest strongly connected component containing the truck's base, discarding unreachable or un-exitable segments.
2. **Graph Balancing via Minimum-Cost Network Flow**: For a directed graph to possess an Eulerian circuit (a path visiting every edge exactly once), every vertex $v \in V$ must be balanced, meaning its in-degree must exactly equal its out-degree: $deg^+(v) - deg^-(v) = 0$. In a real city, this is rarely true. To balance the graph, the truck must revisit certain streets (deadheading). Finding the minimum extra distance required to balance the graph is equivalent to solving the Minimum-Cost Network Flow problem.
    - Nodes with more outgoing than incoming edges ($deg^+ > deg^-$) act as sources of flow.
    - Nodes with more incoming than outgoing edges ($deg^- > deg^+$) act as sinks.
    - The "cost" of sending flow along an edge is the physical length of the road. By computing the optimal flow, we determine exactly which roads must be duplicated (traversed again) to balance the network with the absolute minimum wasted distance.
3. **Circuit Extraction via Hierholzer's Algorithm**: Once the optimal deadhead edges are added to the graph, every node is balanced. Hierholzer's algorithm is then used to construct the final Eulerian circuit by tracing a path through the balanced multigraph, appending smaller sub-tours until every edge has been visited, and returning the exact chronological street schedule.

2. Implementation

The simulator is implemented in Python, leveraging a stack of specialized geospatial and graph theory libraries:

- **Data Acquisition (OSMnx)**: The user's input base coordinates and radius are used to fetch the real road network of Surabaya from OpenStreetMap. A custom Overpass filter ensures only trunk, primary, secondary, tertiary roads, and their corresponding links are downloaded.
- **Graph Processing (NetworkX)**:
    - The raw data is parsed into a nx.MultiDiGraph.
    - nx.strongly_connected_components() isolates the valid operational zone.
    - The imbalance δ(v) = deg+(v) - deg-(v) is calculated for every node. An auxiliary simple directed graph nx.DiGraph is built where the weight of an edge (u, v) is the shortest parallel road length between u and v.
    - nx.min_cost_flow() solves the balancing problem. The resulting flow values dictate how many artificial "revisit" edges must be injected back into the main MultiDiGraph.
- **Route Generation (NetworkX & Pandas)**: nx.eulerian_circuit() generates the sequence of nodes. This sequence is then mapped back to the street names and edge lengths to generate a detailed, ordered pandas DataFrame schedule.
- **Visualization (Streamlit & Folium)**: The interactive web interface allows the user to dynamically adjust the radius and dump truck base. The resulting route is drawn on a map using Folium, with distinct color coding separating primary collection passes from deadhead revisits.

3. Computational Complexity

The overall computational complexity is determined by the three sequential phases of the algorithm:

1. **Graph Construction and SCC Pruning**: Downloading and building the graph takes time proportional to the area size. Finding the strongly connected components using Tarjan's or Kosaraju's algorithm runs in linear time relative to the number of vertices and edges: $O(|V| + |E|)$.
2. **Minimum-Cost Network Flow (Bottleneck)**: This is the most computationally expensive part of the process. NetworkX uses the Network Simplex algorithm to solve the min-cost flow problem. While the worst-case theoretical complexity of Network Simplex can be exponential, its practical performance on sparse, planar-like urban road networks is extremely efficient, typically running in $O(|V| \cdot |E| \log |V|)$ or $O(|V|^3)$. Because the operational radius restricts the graph size (usually < 1000 nodes), this step completes in less than 50 milliseconds.
3. **Hierholzer's Algorithm**: Constructing the Eulerian circuit requires traversing the augmented graph $G_{aug} = (V, E_{aug})$. Since each edge is visited and removed exactly once during the tour construction, this step runs in linear time: $O(|E_{aug}|)$.

Overall Complexity: The time complexity is dominated by the network flow optimization, scaling at approximately $O(|V| \cdot |E| \log |V|)$. Space complexity is $O(|V| + |E|)$ to store the multigraph, the auxiliary flow network, and the geographical coordinates for the map rendering.